進度報告：寫入時依位置放置 KV 的小規模驗證

2026-10-12 ｜ 平台：AMD MI300X ｜ 所有數字都可以由 repo 裡的 CSV 與 run_id 追到原始輸出

老師好，

這是我這幾天的進度。我先照老師 10/7 的建議，在一台機器、一個模型上做了小而完整的驗證。

結論：簡單策略追平了。「寫入時先決定」沒有比延後寫入、成本感知逐出更好。差別主要來自「有沒有 CPU 層、容量夠不夠」和「GPU 忙不忙」，不是寫入的時機。所以「寫到哪裡」不是重點，放置的部分直接用最簡單的做法（寫穿或延後寫入）就可以。

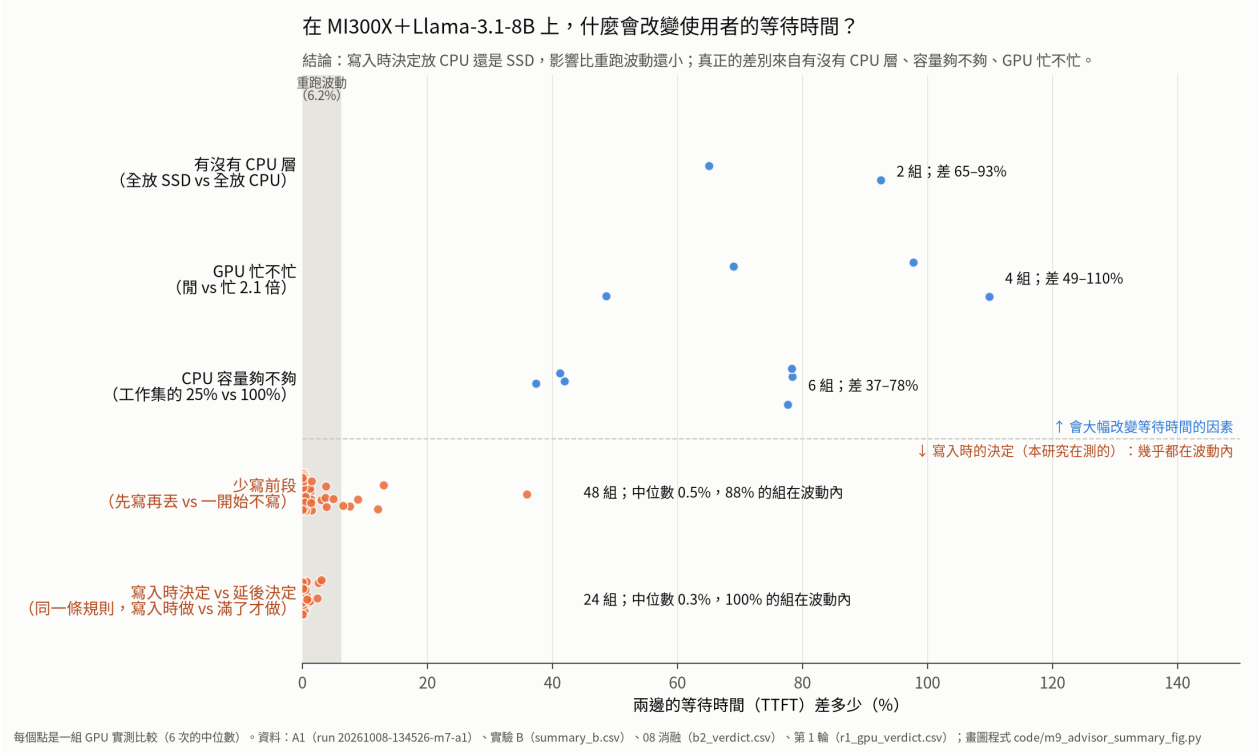


圖 1　總覽：什麼會改變使用者的等待時間。每個點是一組 GPU 實測比較。寫入時的決定（橘色）幾乎都在重跑波動內；有沒有 CPU 層、容量夠不夠、GPU 忙不忙（藍色）才會大幅改變等待時間。

一、驗證的設定

- **機器與模型**：AMD MI300X 單卡；Llama-3.1-8B，BF16；還原一律用 Cake。
- **各層速度**：見表 1。SSD 和 NFS 用限速器模擬，參數取實測最慢值 ×0.9。
- **Context**：單一 session 為 4K–32K。多 session 實驗為 8 個 session，每個最多 4 輪，每輪加 8K（最長 32K）。
- **重算 vs 載入**：見表 2。以一段（512 token）來看，重算要 29–99 毫秒（越後面越貴），從 CPU 載入只要 1.8 毫秒。
- **同一設定重跑的波動是 6.2%**，之後的差距都要大過它才算數。

Cake（最接近的 baseline）有跑通

用 4K–32K × 7 種頻寬（0.33–11.6 GiB/s）共 28 組，檢查原論文的三個趨勢：

表 1　各層頻寬（GiB/s）

層	讀	寫
GPU ↔ CPU（PCIe 實測）	≈50	–
CPU 層	35.4	35.5
本地 SSD	6.98	1.91
NFS	0.33	0.60

表 2　還原一個 32K session 要多久（秒；BF16，KV 約 4 GiB）

儲存層	只算	只載	Cake
本地 SSD 速度	3.76	0.66	0.57
NFS 速度		12.17	2.72

註：本程式的重算比預設 vLLM 慢約 1.7–2 倍（第一階段拿來對照的 vLLM 剛好跑在它的慢路徑上）。把重算加快 2 倍再模擬一次，結論不變，但還沒有 GPU 確認。

1. **context 越長，Cake 比「只算」快越多**：7 種頻寬全部成立。例如本地 SSD 速度下，從 3.1 倍增加到 6.7 倍。
2. **頻寬越快，Cake 比「只載」的好處越小**：最慢和最快兩端成立（NFS 速度 4.5–6.3 倍 → 100 Gbps 0.94–1.27 倍），中間的頻寬不完全照順序。
3. **Cake 不會比「只算」或「只載」慢**：28 組中 27 組成立。例外是 4K × 100 Gbps，Cake 慢 5.6%：只有 8 段，兩條線同時跑的同步開銷大過好處。原論文的 GQA 模型（Llama-3.1-8B）在 32 Gbps 時也只有 0.98 倍。

二、實驗內容

(一) 實驗設計

會比較的做法如表 3（都用 BF16，還原都用 Cake）。

表 3　比較的做法

做法	寫入時	CPU 滿了時	說明
全部都存（純 Cake）	全部寫進 SSD	–	Cake 原本的設定
先寫再丟	先全部寫進 SSD，寫完再刪掉前段（最後留下的和「一開始不寫」一樣，只差前段有沒有先寫過一次）	–	老師點名；看「少寫」本身有沒有用
一開始不寫	前段不寫，其他寫進 SSD	–	老師點名；我原本的設計
寫穿	CPU 和 SSD 各寫一份	刪掉最久沒用的 CPU 那份	業界預設（vLLM、LMCache）
延後寫入	只寫進 CPU	這時才把最久沒用的搬到 SSD	老師點名
延後寫入＋成本感知	只寫進 CPU	這時才把最好重算的（前段）搬到 SSD	兩個簡單方法的組合
依位置放（我的方法）	後段寫進 CPU，前段寫進 SSD	同上	本研究

四組實驗：

- **實驗 A**（單一 session，回答 Q1、Q2）：長度 4K–32K；GPU 分閒、忙 1.5 倍、忙 2.1 倍（同一張卡上跑背景負載，讓重算變慢）；SSD 層分本地 SSD、NFS；使用者分「寫完才回來」和「寫入中就回來」；每格 6 次，共 1,440 次還原。
- **實驗 B**（多 session、CPU 容量有限，回答「簡單策略會不會追平」）：8 個 session 輪流回來，每個最多 4 輪、每輪加 8K；CPU 容量設成工作集的 25%、50%、100%。
- **08 消融**：把對我的方法最有利的條件疊在一起再測：CPU 慢（3.69 GiB/s，vLLM 實測）、容量緊、搬移會卡住請求、一次寫一大段文件。先用模擬篩，再上 GPU。
- **第 1 輪**：模擬篩出唯一有機會的設定，上 GPU 跑 5 個 seed。

（二）結果

1. 省下的寫入有沒有縮短等待？——幾乎沒有（表 4）

「先寫再丟」和「一開始不寫」最後留下的東西一樣，只差前段有沒有先寫過一次。32K 時，「一開始不寫」在 NFS 上少寫了 64 段裡的 51 段（約 80%），在本地 SSD 上少寫 13 段。

表 4　回來請求的等待時間 TTFT（毫秒；GPU 閒；6 次的中位數）

SSD 層	context	寫完才回來		寫入中就回來	
		先寫再丟	一開始不寫	先寫再丟	一開始不寫
本地 SSD	4K	103	95	172	**149（−13%）**
	8K	156	156	302	291
	16K	292	288	577	582
	32K	533	580	1,192	1,195
NFS	4K	250	340	345	345
	8K	525	533	672	**591（−12%）**
	16K	1,206	1,189	1,313	1,248
	32K	2,746	2,745	3,114	**2,910（−6.5%）**

粗體：差距大於重跑波動 6.2%，而且 6 次重跑的範圍不重疊；其他格的差距都在波動內。GPU 忙的 32 組見附錄表 A1。

- **寫完才回來**：兩種做法幾乎一樣。例如 NFS、32K，少寫了 80% 的資料，等待時間還是 2,746 vs 2,745 毫秒。
- **寫入中就回來**：使用者在上一輪的資料還沒寫完時就回來，硬碟要同時寫和讀，所以讀變慢（例：本地 SSD、32K，從 533 毫秒變成 1,192 毫秒）。這時「一開始不寫」雖然寫得比較少，但大多數情況下，回來時寫入一樣還沒結束，硬碟一樣被佔著，所以只有 3 組明顯變快（6.5–13%）。
- **GPU 忙的時候**（另外 32 組）：兩種做法的差距都在 ±3.8% 以內，沒有一組明顯變快。
- **結論**：少寫本身不會讓等待變短，只有少寫到讓寫入在使用者回來前就寫完，才有一點幫助。

2. GPU 閒的時候，不存前段沒差；GPU 忙的時候，代價很大（表 5、圖 2）

表 5　「一開始不寫」比「全部都存」慢多少（正值＝較慢）

SSD 層	context	GPU 閒	GPU 忙 1.5 倍	GPU 忙 2.1 倍
本地 SSD	16K	−1.1%（不顯著）	+36.2%	+52.0%
	32K	+7.1%（不顯著）	+29.2%	+66.4%
NFS	16K	−0.8%（不顯著）	+13.2%	+24.9%
	32K	+0.9%（不顯著）	+14.2%	+30.5%

- **GPU 閒**：Cake 本來就會重算前段，所以前段存不存都一樣。
- **GPU 忙**：「全部都存」可以讓 Cake 少算幾段、改從 SSD 搬（本地 SSD、32K 時，從重算 13 段降到 7 段）。「一開始不寫」沒有備份，只能硬算 13 段，所以 16K 以上慢 13–66%。
- 所以我把設計改成「每段都存，只改放的位置」。
- 圖 2 把每次還原拆成「GPU 重算」和「I/O 載入」兩條線，可以直接看到省下的 I/O 和增加的計算。

3. 和延後寫入、成本感知逐出比較：簡單策略追平（表 6）

實驗 B（8 個 session）：

- 依位置放 vs 延後寫入＋成本感知：所有設定只差 −3.0%～+0.3%。
- CPU 容量 25% 時，依位置放反而比最好的簡單策略慢 29%；50% 時慢 3.7%；100% 時差 0.4%。

表 6　「寫入時決定」vs 簡單策略、延後版（GPU 實測，共 36 組比較）

實驗	比較組數	贏簡單策略 ≥5%	贏延後版 ≥5%	兩邊都贏 ≥5%
實驗 B（CPU 容量 25 ／ 50 ／ 100%）	15	0	0	0
08 消融（有利條件疊在一起）	16	1	5	0
第 1 輪（5 個 seed）	5	0	0	0
合計	36	1	5	**0**

簡單策略：寫穿、延後寫入、成本感知逐出等，取最好的。延後版：同一條規則（前段放 SSD、後段放 CPU），但等 CPU 滿了才做。

- 唯一贏簡單策略的那一組（快 18.9%），只比延後版快 0.7%，換一個 seed 就不見了：贏的是規則本身，不是「寫入時」。
- 贏延後版的 5 組，都沒贏簡單策略（−4.8%～+0.4%）。它們都在「搬移會卡住請求」的設定，那裡寫穿本來就最快。
- 之後又從多個角度（多人同時使用、真實 vLLM、真實 trace、換模型等）和新的 seed 重測，結論都一樣。

（三）小結

- **為什麼延後版一樣好**：決定哪幾段放 SSD，看的是「這段在第幾頁」。這個資訊不會變，等 CPU 滿了要搬的時候一樣知道。所以「CPU 滿了才把前段搬去 SSD」，最後的擺法和「一寫進來就把前段放 SSD」一模一樣，速度也一樣。

- **真正影響等待時間的**：資料在不在 CPU（CPU 比 SSD 快很多）、CPU 放不放得下、GPU 忙不忙（圖 1）。寫入的時機幾乎沒有影響。
- **寫入時唯一要注意的**：不要一開始就不存。沒存的段之後就沒了，GPU 忙的時候只能重算，會慢 13–66%（表 5）。所以寫入時每一段都存，等 CPU 滿了再搬到 SSD 就好。

附圖

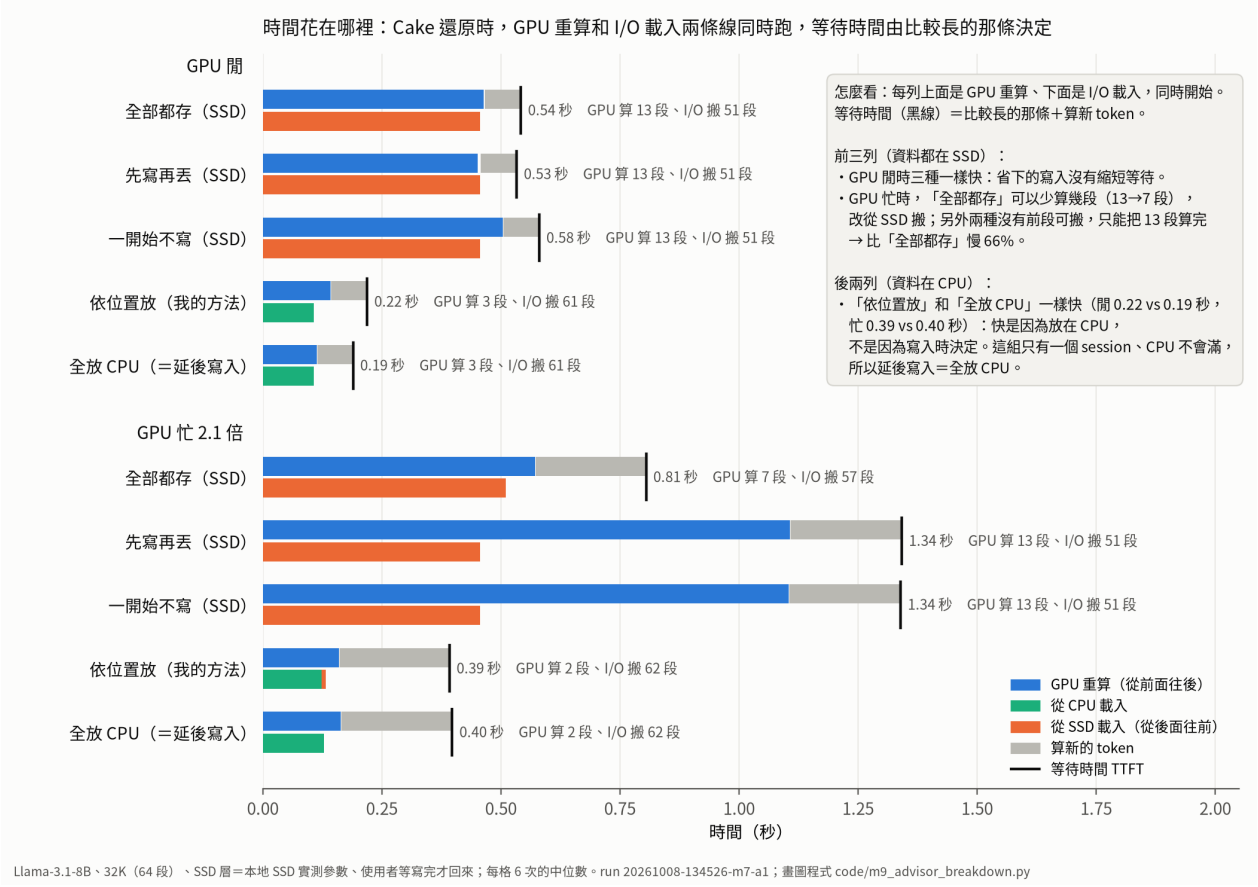


圖 2　時間花在哪裡（32K、本地 SSD、使用者等寫完才回來）。每列上面是 GPU 重算、下面是 I/O 載入，兩條同時跑，等待時間由比較長的那條決定。前三列資料在 SSD：GPU 閒時一樣快，GPU 忙時沒存前段的只能把 13 段算完。後兩列資料在 CPU：「依位置放」和「全放 CPU（＝延後寫入）」一樣快。

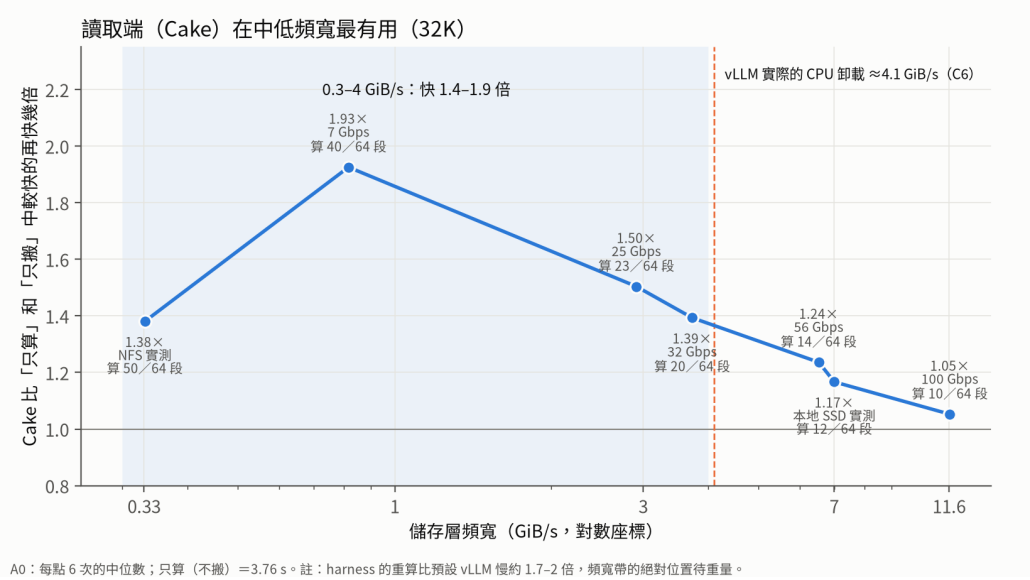


圖 3　讀取端（Cake）在中低頻寬（0.3–4 GiB/s）最有用：比「只算」或「只載」中較快的再快 1.4–1.9 倍；11.6 GiB/s 時只快 5%。vLLM 實際的 CPU 卸載約 4.1 GiB/s，正好落在這個範圍。

附錄：完整數字

表 A1　先寫再丟 vs 一開始不寫，GPU 忙時的等待時間 TTFT（毫秒；6 次的中位數）

GPU	SSD 層	context	寫完才回來		寫入中就回來	
			先寫再丟	一開始不寫	先寫再丟	一開始不寫
忙 1.5 倍	本地 SSD	4K	145	146	198	198
		8K	203	203	357	352
		16K	432	445	717	720
		32K	821	828	1,399	1,404
	NFS	4K	414	412	398	400
		8K	837	826	825	823
		16K	1,777	1,798	1,782	1,787
		32K	4,100	4,083	4,234	4,078
忙 2.1 倍	本地 SSD	4K	251	252	272	271
		8K	358	354	515	496
		16K	742	739	912	916
		32K	1,342	1,339	1,654	1,652
	NFS	4K	664	662	655	658
		8K	1,318	1,315	1,318	1,322
		16K	2,814	2,807	2,800	2,802
		32K	5,975	5,996	5,992	5,987

32 組的差距都在 ±3.8% 以內，沒有一組明顯變快。

表 A2　表 6 的 36 組比較（回來請求 TTFT 中位數，秒）

#	實驗	設定	寫入時策略	最好的簡單策略	TTFT（秒） 寫入時	簡單	延後版	寫入時決定快多少 vs 簡單	vs 延後版
1	B	對話型、本地、25%、s0	S5	S4	1.441	1.235	1.428	−16.7%	−0.9%
2	B	對話型、本地、50%、s0	S5	S1	1.134	1.092	1.129	−3.8%	−0.4%
3	B	對話型、NFS、25%、s0、SSD 50%	S5	S4	1.932	1.509	1.929	−28.0%	−0.2%
4	B	對話型、NFS、50%、s0、SSD 50%	S5	S1	1.381	1.331	1.380	−3.8%	−0.0%
5	B	對話型、NFS、25%、s0	S5	S4	1.933	1.494	1.934	−29.4%	+0.1%
6	B	對話型、NFS、50%、s0	S5	S1	1.381	1.331	1.384	−3.7%	+0.3%
7	B	對話型、NFS、100%、s0	S5	S2b	1.088	1.083	1.085	−0.4%	−0.3%
8	B	對話型、NFS、25%、s1	S5	S1	1.421	1.071	1.380	−32.6%	−3.0%
9	B	對話型、NFS、50%、s1	S5	S4	1.084	1.073	1.082	−1.0%	−0.2%
10	B	對話型、NFS、25%、s2	S5	S4+	1.436	1.436	1.436	−0.0%	−0.0%
11	B	對話型、NFS、50%、s2	S5	S4+P	1.129	1.099	1.129	−2.8%	+0.0%
12	B	對話型、NFS、25%、s0、間隔 2s	S5	S1	1.828	1.487	1.782	−22.9%	−2.6%
13	B	對話型、NFS、50%、s0、間隔 2s	S5	S1	1.332	1.326	1.329	−0.4%	−0.2%
14	B	對話型、NFS、25%、s0、CPU 3.69	S5	S4+P	1.835	1.637	1.823	−12.1%	−0.7%
15	B	對話型、NFS、50%、s0、CPU 3.69	S5	S4	1.436	1.437	1.438	+0.1%	+0.2%
16	08	對話型、CPU 35.4、NFS、25%、s0	S5	S1	3.450	1.555	3.151	−121.9%	−9.5%
17	08	對話型、CPU 35.4、NFS、25%、s0	S5c	S1	3.016	1.555	3.151	−94.0%	+4.3%
18	08	對話型、CPU 35.4、NFS、50%、s0	S5	S1	2.908	1.332	2.733	−118.3%	−6.4%
19	08	對話型、CPU 35.4、NFS、50%、s0	S5c	S1	1.360	1.332	2.733	−2.1%	**+50.2%**

#	實驗	設定	寫入時	最好的簡單	寫入時	簡單	延後版	vs 簡單	vs 延後版
20	08	文件型、CPU 3.69、NFS、25%、s0	S5	S2b	2.161	2.061	2.851	−4.8%	**+24.2%**
21	08	文件型、CPU 3.69、NFS、25%、s0	S5L	S2b	2.851	2.061	2.851	−38.3%	+0.0%
22	08	文件型、CPU 3.69、NFS、25%、s0	S5c	S2b	2.063	2.061	2.851	−0.1%	**+27.6%**
23	08	文件型、CPU 3.69、NFS、50%、s0	S5	S2b	1.488	1.389	1.134	−7.1%	−31.2%
24	08	文件型、CPU 3.69、NFS、50%、s0	S5L	S2b	1.127	1.389	1.134	**+18.9%**	+0.7%
25	08	文件型、CPU 3.69、NFS、50%、s0	S5c	S2b	1.400	1.389	1.134	−0.8%	−23.4%
26	08	文件型、CPU 3.69、NFS、25%、s1	S5	S2b	2.178	2.079	2.851	−4.8%	**+23.6%**
27	08	文件型、CPU 3.69、NFS、25%、s1	S5L	S2b	2.764	2.079	2.851	−33.0%	+3.1%
28	08	文件型、CPU 3.69、NFS、25%、s1	S5c	S2b	2.070	2.079	2.851	+0.4%	**+27.4%**
29	08	文件型、CPU 3.69、NFS、50%、s1	S5	S1	1.193	0.891	0.952	−33.9%	−25.3%
30	08	文件型、CPU 3.69、NFS、50%、s1	S5L	S1	0.952	0.891	0.952	−6.8%	−0.0%
31	08	文件型、CPU 3.69、NFS、50%、s1	S5c	S1	1.253	0.891	0.952	−40.6%	−31.6%
32	第 1 輪	文件型、CPU 11.6、本地、50%、s0	S5L	S4L	0.456	0.442	0.461	−3.1%	+1.2%
33	第 1 輪	文件型、CPU 11.6、本地、50%、s1	S5L	S4L	0.422	0.425	0.424	+0.5%	+0.5%
34	第 1 輪	文件型、CPU 11.6、本地、50%、s2	S5L	S4	0.457	0.454	0.453	−0.6%	−0.7%
35	第 1 輪	文件型、CPU 11.6、本地、50%、s3	S5L	S1	0.443	0.428	0.432	−3.4%	−2.4%
36	第 1 輪	文件型、CPU 11.6、本地、50%、s4	S5L	S4	0.424	0.425	0.425	+0.1%	+0.2%

設定欄依序是：負載型態、CPU 層頻寬（GiB/s，沒寫的是 35.4）、SSD 層（本地 SSD 或 NFS）、CPU 容量（佔工作集的比例）、workload seed。08 的 16 組都是「搬移會卡住請求」的設定；「SSD 50%」＝ SSD 容量也有限。

正值＝寫入時決定比較快；粗體＝快 ≥5%。「兩邊都快 ≥5%」的組數是 0。

實驗 B 是 8 個 session 的對話型負載（第一階段）；08 消融刻意疊加對寫入時決定有利的條件；第 1 輪是模擬篩出的唯一設定。

寫入時策略：S5 ＝依位置放（後段放 CPU、前段放 SSD，CPU 滿了照成本感知搬）；S5L ＝同樣依位置放，但 CPU 滿了先搬最久沒用的 session；S5c ＝全部放 CPU，另外在寫入時把前段先複製一份到 SSD。

延後版：實驗 B 為 S4+（只寫 CPU，滿了才把前段搬到 SSD）；08 與第 1 輪為 S4B（只寫 CPU，滿了才照同一條位置規則搬）。

簡單策略：S1 ＝寫穿＋ LRU；S2b ＝寫穿＋成本感知逐出；S4 ＝延後寫入＋ LRU；S4+ ＝延後寫入＋成本感知；S4+P ＝延後寫入＋ Pensieve 式（成本 × 閒置時間）；S4L ＝延後寫入，搬最久沒用的 session、從前段開始。

資料來源：表 4、A1：`results/m7_write_policy_mi300x/summary_a1_pairs.csv`（run 20261008-134526-m7-a1）；表 2：`a0.csv`（run 20261008-133544-m7-a0）；表 5：07 報告 §5；表 6、A2：`summary_b.csv`、`b2_verdict.csv`、`b2_summary.csv`、`r1_gpu_verdict.csv`（整理：`code/m9_advisor_table36.py`）；圖：`code/m9_advisor_summary_fig.py`（run 20261013-030010-advisor-summary）、`m9_advisor_breakdown.py`（run 20261013-025740-advisor-breakdown）、`m9_advisor_figs.py`（run 20261012-152226-advisor-figs）。